

Blockchain

Fondamentaux, Sécurité et Applications

Programme de la formation

Module	Thème	Contenu clé
1	Fondamentaux de la Blockchain	Architecture, blocs, chaînes, décentralisation
2	Cryptographie et sécurité	Hash, signatures, Merkle trees, consensus
3	Plateformes et écosystème	Bitcoin, Ethereum, Hyperledger, Solana
4	Smart Contracts	Solidity, déploiement, sécurité, audit
5	Applications sectorielles	DeFi, Supply Chain, Identité, DAO
6	Défis et limitations	Scalabilité, régulation, énergie, vie privée

Objectifs pédagogiques

- Evaluer les principes fondamentaux de la Blockchain et les services de sécurité associés (immutabilité, traçabilité, décentralisation, transparence)
- Comprendre les types d'applications adaptées et comment la Blockchain transforme les secteurs (finance, industrie, commerce, administration)
- Analyser les mécanismes de sécurité : cryptographie, consensus décentralisé, validation des transactions
- Appliquer la Blockchain à des cas d'usage réels en concevant des solutions sécurisées et efficaces
- Explorer les défis et limitations : réglementations, performances, évolutivité

Module 1

Fondamentaux de la Blockchain

Qu'est-ce que la Blockchain ?

Un **registre distribué** (Distributed Ledger) qui enregistre des transactions de manière :

- **Décentralisée** : pas d'autorité centrale unique
- **Immuable** : une fois écrit, un bloc ne peut plus être modifié
- **Transparente** : tous les participants peuvent vérifier l'historique
- **Sécurisée** : protégée par la cryptographie et le consensus

“ La Blockchain est une chaîne de blocs liés cryptographiquement, répliquée sur un réseau de nœuds pair-à-pair. ”

Un changement de paradigme

Modèle centralisé (traditionnel)

```
Client A ┌  
Client B ┤ — Serveur Central (banque, état, notaire)  
Client C └  
  
Point unique de défaillance  
Point unique de confiance
```

Un changement de paradigme

Modèle décentralisé (Blockchain)

Nœud A ↔ Nœud B



Nœud C ↔ Nœud D



Nœud E ↔ Nœud F

Chaque nœud détient une copie complète

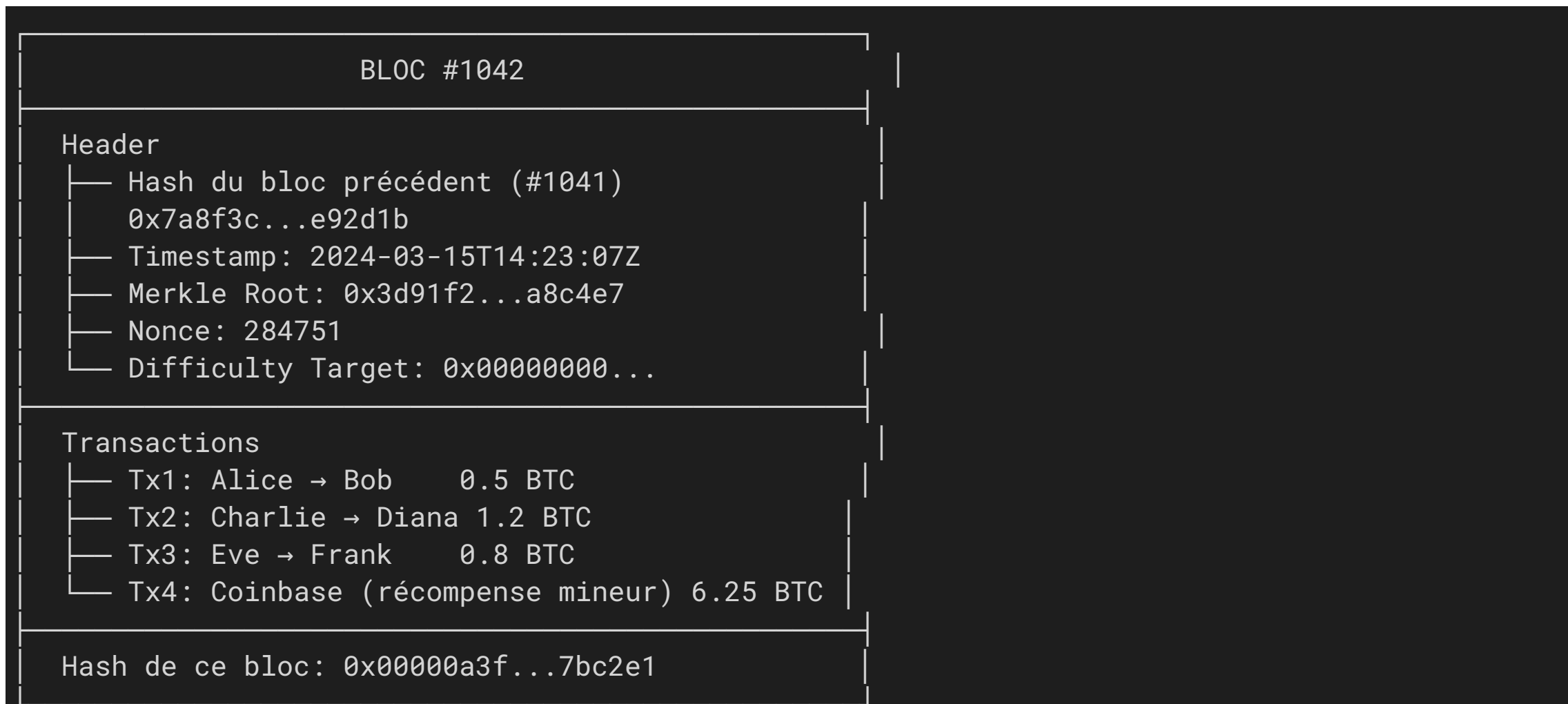
Analogie : le registre du village

Imaginez un village sans banque ni notaire :

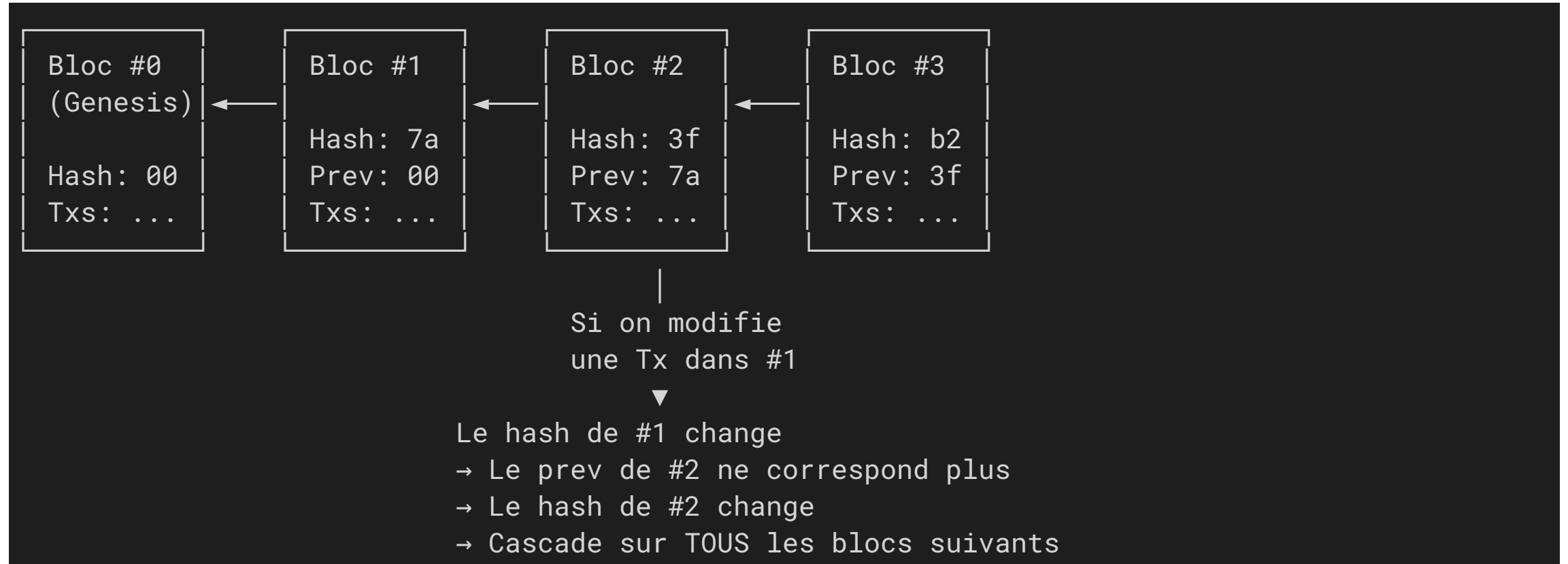
1. **Ancien système** : un seul cahier chez le maire → risque de fraude, perte, corruption
2. **Système Blockchain** :
 - Chaque habitant possède une **copie identique** du cahier
 - Quand une transaction a lieu, **tout le monde l'inscrit** simultanément
 - Les pages sont **numérotées et scellées** à la cire (hash cryptographique)
 - Modifier une page ancienne obligerait à **refaire toutes les pages suivantes...** sur **toutes les copies**

“ C'est mathématiquement et pratiquement impossible à falsifier. ”

Anatomie d'un bloc



La chaîne de blocs

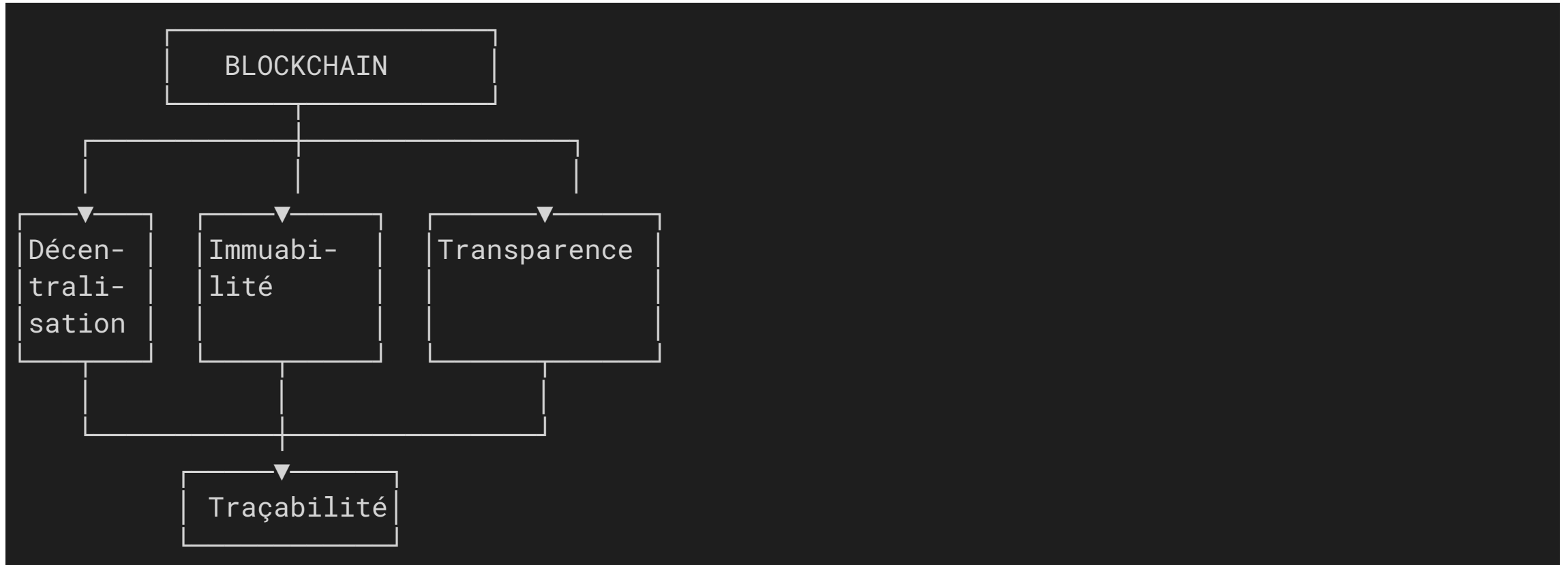


“ **Immuabilité** : modifier un bloc passé invalide toute la chaîne. ”

Types de Blockchain

Caractéristique	Publique	Privée	Consortium
Accès	Ouvert à tous	Restreint (1 org.)	Restreint (N org.)
Lecture	Tout le monde	Membres autorisés	Membres autorisés
Écriture	Tout le monde	Administrateur	Membres validateurs
Consensus	PoW, PoS	RAFT, PBFT	PBFT, PoA
Performance	~15-30 TPS	~1000+ TPS	~100-1000 TPS
Exemples	Bitcoin, Ethereum	Hyperledger Fabric	R3 Corda, Quorum
Cas d'usage	Crypto, DeFi	Entreprise interne	Inter-entreprises

Les quatre piliers de la Blockchain



Les quatre piliers de la Blockchain

- **Décentralisation** : aucun point unique de contrôle ni de défaillance
- **Immuabilité** : les données enregistrées ne peuvent pas être modifiées
- **Transparence** : l'historique est vérifiable par tous les participants
- **Traçabilité** : chaque transaction est horodatée et liée à son origine

Le réseau pair-à-pair (P2P)

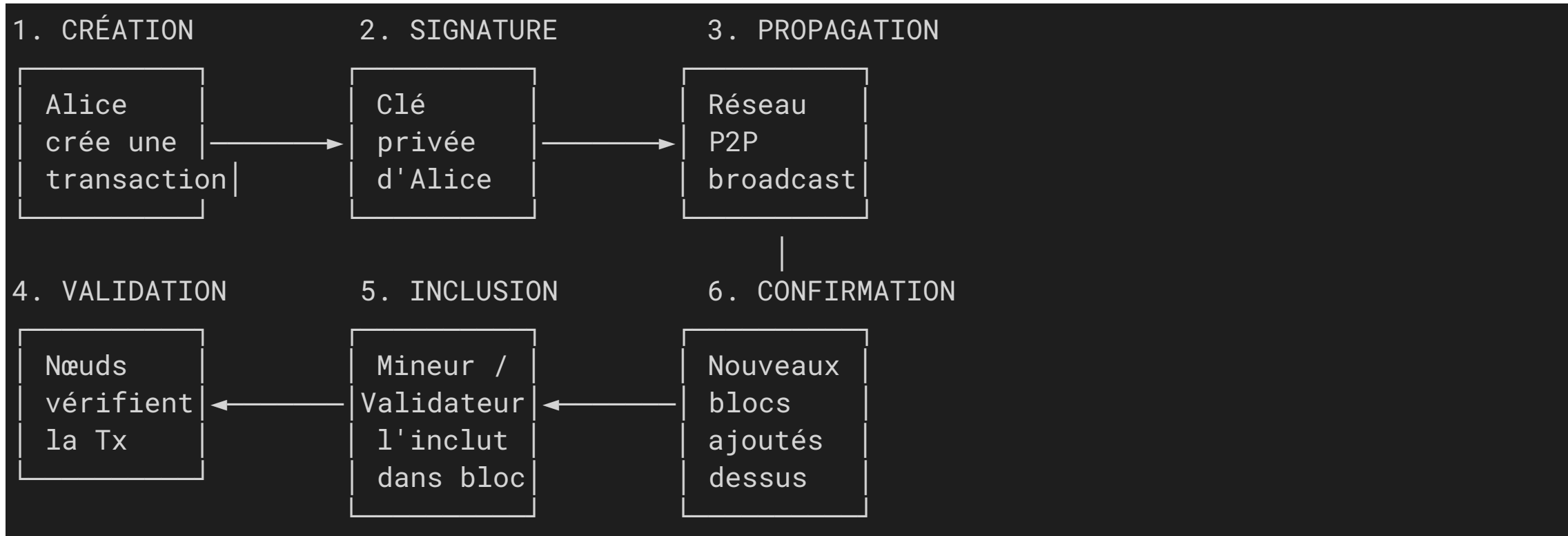
Rôles des nœuds dans le réseau :

Type de nœud	Rôle	Exemple
Full Node	Stocke toute la blockchain, valide tout	Bitcoin Core
Light Node	Stocke les headers, vérifie via SPV	Wallets mobiles
Mining Node	Full node + création de nouveaux blocs	Pool de minage
Archive Node	Full node + tous les états historiques	Etherscan, Infura
Validator	Valide et propose des blocs (PoS)	Stakers Ethereum

“ Chaque nœud complet détient une copie identique de l'intégralité de la blockchain.

”

Cycle de vie d'une transaction



Le Mempool

Avant d'être incluse dans un bloc, chaque transaction attend dans le **mempool** :

Transaction Pool (Mempool)

Tx: Alice→Bob	0.5 BTC	Fee: 0.0005	← haute priorité
Tx: Charlie→Dan	1.0 BTC	Fee: 0.0003	
Tx: Eve→Frank	0.1 BTC	Fee: 0.0001	← basse priorité
Tx: Grace→Hank	2.0 BTC	Fee: 0.0004	
... potentiellement des milliers			

↓
▼ Le mineur/validateur sélectionne
les Tx les plus rentables (fees)

“ Les mineurs/validateurs priorisent les transactions avec les frais (fees) les plus élevés. ”

Le Genesis Block

Le **bloc Genesis** (Bloc #0) est le premier bloc de toute blockchain :
Bitcoin Genesis Block (3 janvier 2009) :

```
Hash:      000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
Prev Hash: 0000000000000000000000000000000000000000000000000000000000000000
Timestamp: 2009-01-03 18:15:05
Nonce:     2083236893
```

Message inclus par Satoshi Nakamoto :

“ "The Times 03/Jan/2009 Chancellor on brink of second bailout for banks" ”

Ce message prouve la date de création et symbolise la motivation derrière Bitcoin : une alternative au système bancaire centralisé.

Blockchain vs Base de données traditionnelle

Critère	Base de données	Blockchain
Architecture	Centralisée (client-serveur)	Distribuée (P2P)
Confiance	L'opérateur de la BDD	Le protocole mathématique
Modification	CRUD complet	Append-only (ajout seul)
Admin	DBA avec tous les droits	Consensus du réseau
Performance	Milliers de TPS	~15-65 TPS (publique)
Coût	Serveurs centralisés	Coût par transaction (gas)
Transparence	Opaque par défaut	Transparente par défaut
Cas d'usage	Apps classiques, CRUD	Confiance sans tiers, audit

“ La Blockchain n'est **pas** un remplacement des bases de données. C'est un outil pour des cas où la **confiance décentralisée** est nécessaire. ”

Quand utiliser (ou non) la Blockchain ?

Utiliser la Blockchain quand :

- Plusieurs parties ne se font pas confiance mutuellement
- On a besoin d'un historique immuable et vérifiable
- La désintermédiation (supprimer le tiers de confiance) a de la valeur
- La traçabilité de bout en bout est critique
- L'automatisation via smart contracts apporte de l'efficacité

NE PAS utiliser la Blockchain quand :

- Une seule entité contrôle les données → base de données classique
- Les données doivent être modifiables ou supprimables (RGPD)
- Les performances temps réel sont critiques (< 1s)
- Le volume de données est très important (stockage on-chain coûteux)
- La confidentialité totale est requise (blockchain publique = transparent)

Module 2

Cryptographie et Mécanismes de Sécurité

Les piliers cryptographiques

La sécurité de la Blockchain repose sur trois mécanismes fondamentaux :

SÉCURITÉ BLOCKCHAIN		
Fonctions de hachage	Cryptographie asymétrique	Protocoles de consensus
SHA-256 Keccak-256 Merkle Trees	Clé publique Clé privée Signatures ECDSA	PoW, PoS PBFT, DPoS

Fonctions de hachage

Une fonction de hachage transforme une entrée de taille arbitraire en une sortie de taille fixe :

```
"Hello"      → 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c  
"Hello!"    → 334d016f755cd6dc58c53a86e183882f8ec14f52  
"Hello!!"   → 7a29a238913c5b81c166c9d95d8f079e60094b79
```



Un seul caractère change = hash complètement différent

Propriétés essentielles :

- **Déterministe** : même entrée = même sortie, toujours
- **Rapide** à calculer
- **Effet avalanche** : un petit changement modifie radicalement le hash
- **Résistance à la préimage** : impossible de retrouver l'entrée depuis le hash
- **Résistance aux collisions** : impossible de trouver deux entrées avec le même hash

SHA-256 — L'algorithme de Bitcoin

```
import hashlib

# Hachage simple
message = "Blockchain Formation Utopios"
hash_result = hashlib.sha256(message.encode()).hexdigest()
print(hash_result)
# → "a7f3c8d2e1b5..." (64 caractères hexadécimaux = 256 bits)

# Démonstration de l'effet avalanche
msg1 = "Transaction: Alice envoie 1 BTC à Bob"
msg2 = "Transaction: Alice envoie 2 BTC à Bob"

h1 = hashlib.sha256(msg1.encode()).hexdigest()
h2 = hashlib.sha256(msg2.encode()).hexdigest()

print(f"Hash 1: {h1}")
print(f"Hash 2: {h2}")
# Les deux hash sont totalement différents malgré un seul chiffre changé
```

Proof of Work — Le minage

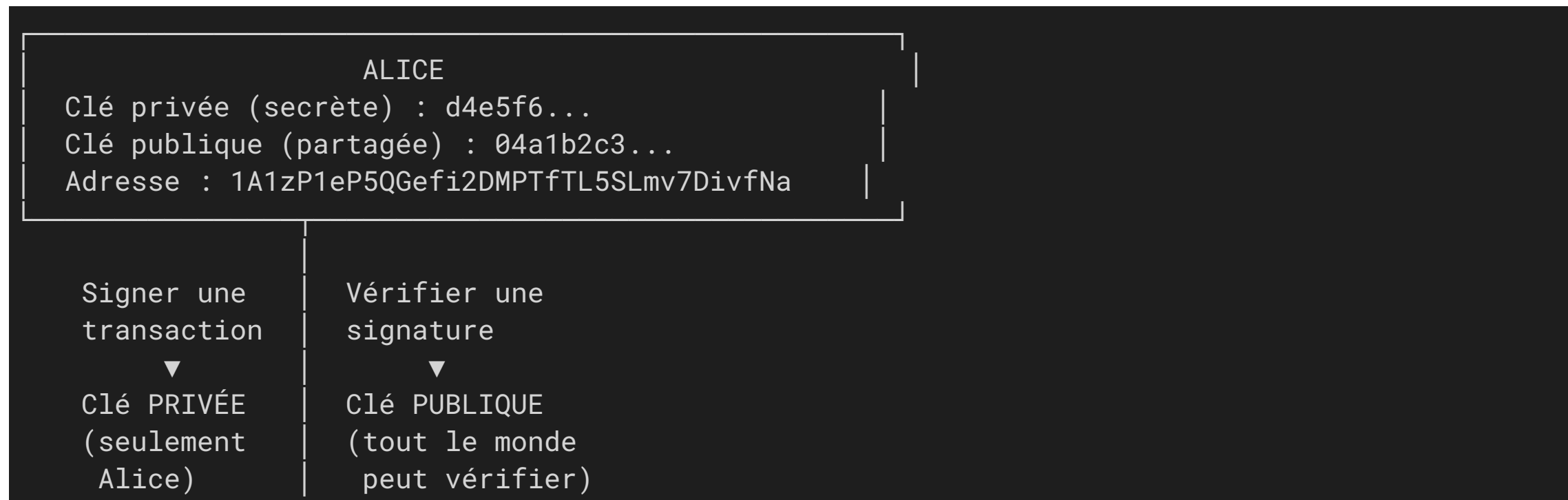
Le minage consiste à trouver un **nonce** tel que le hash du bloc commence par un certain nombre de zéros :

```
import hashlib

def mine_block(data, difficulty):
    """Simule le minage d'un bloc"""
    prefix = "0" * difficulty
    nonce = 0
    while True:
        text = f"{data}{nonce}"
        hash_result = hashlib.sha256(text.encode()).hexdigest()

        if hash_result.startswith(prefix):
            return nonce, hash_result
        nonce += 1
# Difficulté 4 = le hash doit commencer par "0000"
nonce, hash_val = mine_block("Bloc de formation", 4)
print(f"Nonce trouvé: {nonce}")
print(f"Hash: {hash_val}")
# Hash: 0000a7f3c8d2e1b5...
```

Cryptographie asymétrique (clés publique/privée)



- **Clé privée** → signe les transactions (preuve d'identité)
- **Clé publique** → dérivée de la clé privée, vérifie les signatures
- **Adresse** → dérivée de la clé publique, identifiant public

“ Relation à sens unique : privée → publique → adresse (impossible de remonter). ”

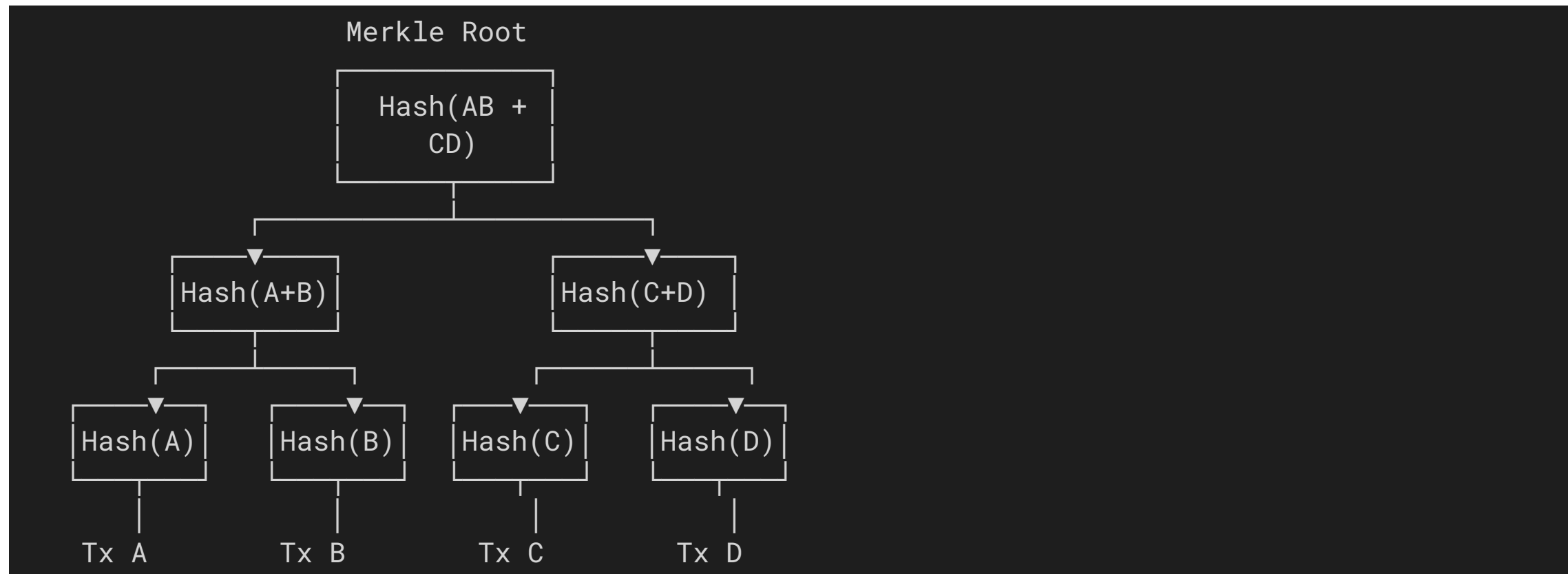
ECDSA – Signature des transactions

```
# Illustration conceptuelle (simplifié)
from ecdsa import SigningKey, SECP256k1
# Génération de la paire de clés
private_key = SigningKey.generate(curve=SECP256k1)
public_key = private_key.get_verifying_key()
# Signer une transaction
transaction = b"Alice envoie 1 BTC a Bob"
signature = private_key.sign(transaction)
# N'importe qui peut vérifier avec la clé publique
is_valid = public_key.verify(signature, transaction)
print(f"Signature valide: {is_valid}") # True

# Tentative de falsification
fake_tx = b"Alice envoie 100 BTC a Mallory"
try:
    public_key.verify(signature, fake_tx)
except:
    print("INVALIDE: la signature ne correspond pas")
```

Arbre de Merkle

Un **Merkle Tree** permet de vérifier l'intégrité de milliers de transactions avec un seul hash :



Avantage : pour vérifier que Tx B est dans le bloc, il suffit de Hash(A), Hash(CD) et la Merkle Root. Pas besoin de télécharger toutes les transactions.

Protocoles de consensus

Le consensus est le mécanisme par lequel le réseau s'accorde sur l'état de la blockchain :

Protocole	Mécanisme	Énergie	TPS	Utilisé par
PoW	Puissance de calcul	Très élevée	~7-30	Bitcoin
PoS	Mise en jeu de tokens	Faible	~30-100k	Ethereum 2.0
DPoS	Vote de délégués	Faible	~1000+	EOS, TRON
PoA	Identité des validateurs	Très faible	~1000+	Réseaux privés
PBFT	Vote 2/3 majorité	Faible	~1000+	Hyperledger
PoH	Preuve de temps	Faible	~65000	Solana

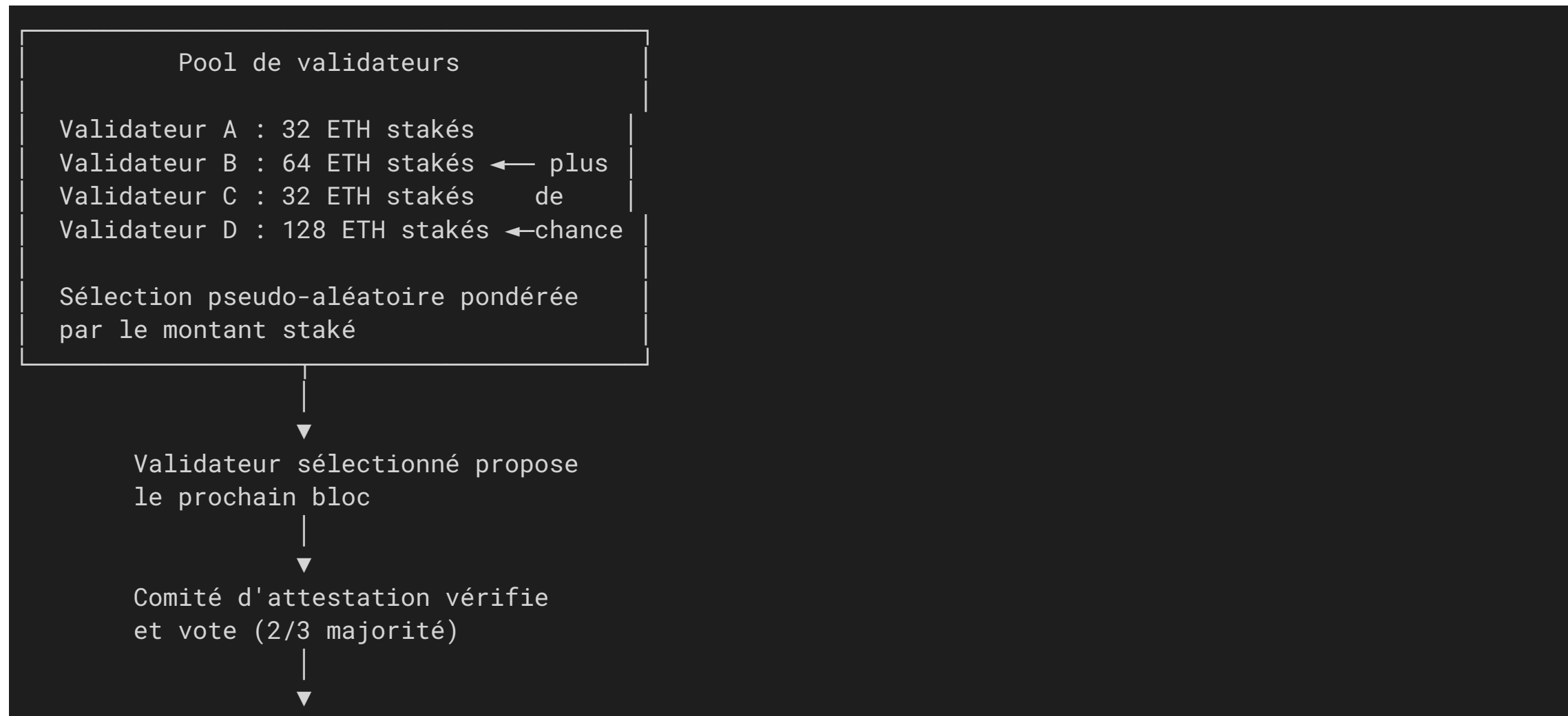
Proof of Work (PoW) en détail

1. Un mineur collecte les transactions du mempool
2. Il construit un bloc candidat
3. Il cherche un nonce tel que :
 $\text{hash}(\text{bloc} + \text{nonce}) < \text{target_difficulty}$

Nonce 0: hash = "8f3a2b..." ❌ trop grand
Nonce 1: hash = "c7e91d..." ❌
Nonce 2: hash = "a1f4e8..." ❌
...
Nonce 284751: hash = "00000a3f..." ✅ TROUVÉ !
4. Le mineur diffuse le bloc au réseau
5. Les autres nœuds vérifient (instantané)
6. Le bloc est ajouté à la chaîne
7. Le mineur reçoit la récompense (6.25 BTC en 2024)

“ **Asymétrie** : difficile à trouver (minutes/heures), trivial à vérifier (millisecondes). ”

Proof of Stake (PoS) – Ethereum 2.0

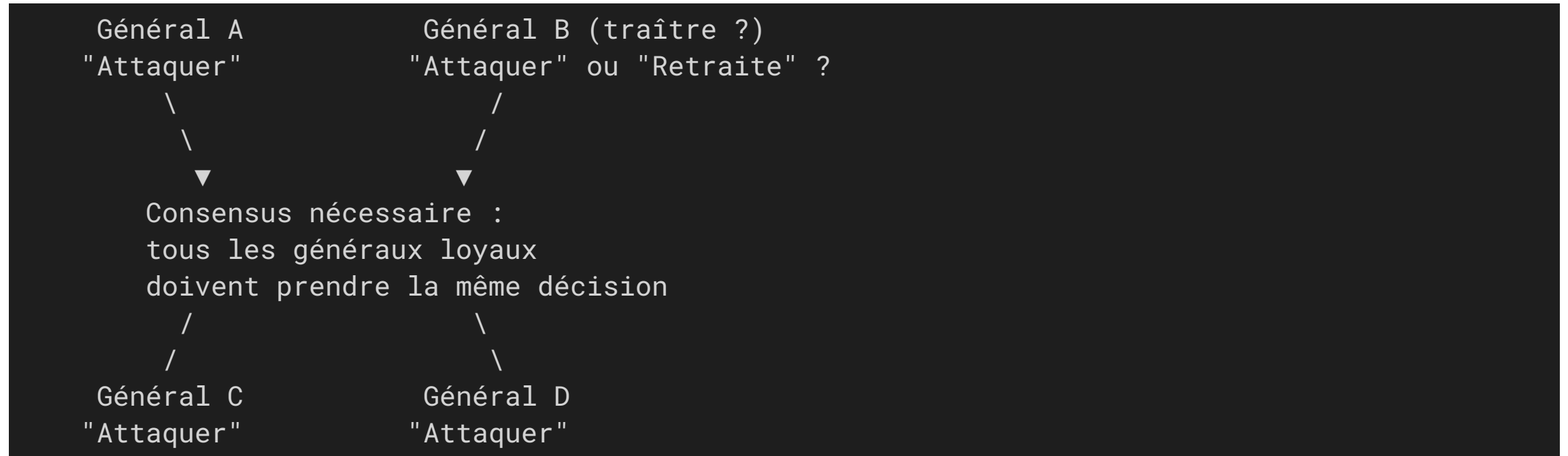


Attaques et défenses

Attaque	Description	Défense
51%	Contrôler la majorité du réseau	Coût prohibitif sur les grands réseaux
Double Spending	Dépenser deux fois les mêmes fonds	Attendre plusieurs confirmations
Sybil	Créer de faux nœuds en masse	PoW/PoS rend chaque identité coûteuse
Eclipse	Isoler un nœud du réseau réel	Connexions diversifiées, checkpoints
Selfish Mining	Garder des blocs secrets	Protocoles de détection
Front-running	Voir les Tx en mempool et passer devant	Commit-reveal, Flashbots
Replay Attack	Rejouer une Tx sur un autre réseau	Chain ID, protection intégrée

Le problème des généraux byzantins

Problème : comment des généraux (nœuds) qui ne se font pas confiance et communiquent par messages (réseau) peuvent-ils s'accorder sur une action commune, sachant que certains généraux et messagers peuvent être traîtres ?



“ La Blockchain résout ce problème via ses protocoles de consensus (PoW, PoS, PBFT), permettant un accord même avec jusqu'à 1/3 de nœuds malveillants. ”

Module 3

Plateformes et Écosystème Blockchain

Évolution historique

2008 — Whitepaper Bitcoin (Satoshi Nakamoto)
2009 — Genesis Block Bitcoin (3 janvier)
2013 — Whitepaper Ethereum (Vitalik Buterin)
2015 — Lancement Ethereum – Smart Contracts
2015 — Hyperledger (Linux Foundation)
2017 — Boom des ICO – ERC-20 tokens
2018 — DeFi commence (MakerDAO, Uniswap)
2020 — DeFi Summer – TVL explose
2020 — Lancement Ethereum Beacon Chain (PoS)
2021 — NFT mania – Bepple, CryptoPunks
2022 — The Merge – Ethereum passe au PoS
2023 — Layer 2 : Arbitrum, Optimism, zkSync
2024 — Bitcoin ETF approuvé – institutionnalisation
2024 — RWA (Real World Assets) tokenisés

Bitcoin — La première Blockchain

Architecture Bitcoin :

Caractéristique	Valeur
Consensus	Proof of Work (SHA-256)
Temps de bloc	~10 minutes
Taille de bloc	1 Mo (~4 Mo avec SegWit)
Supply max	21 millions BTC
Halving	Tous les 210 000 blocs (~4 ans)
Langage de script	Bitcoin Script (non Turing-complet)
TPS	~7 transactions/seconde

Modèle UTXO (Unspent Transaction Output) :

Chaque transaction consomme des sorties non dépensées et en crée de nouvelles. Pas de notion de "solde" : le solde est la somme des UTXO possédés.

Ethereum — La Blockchain programmable

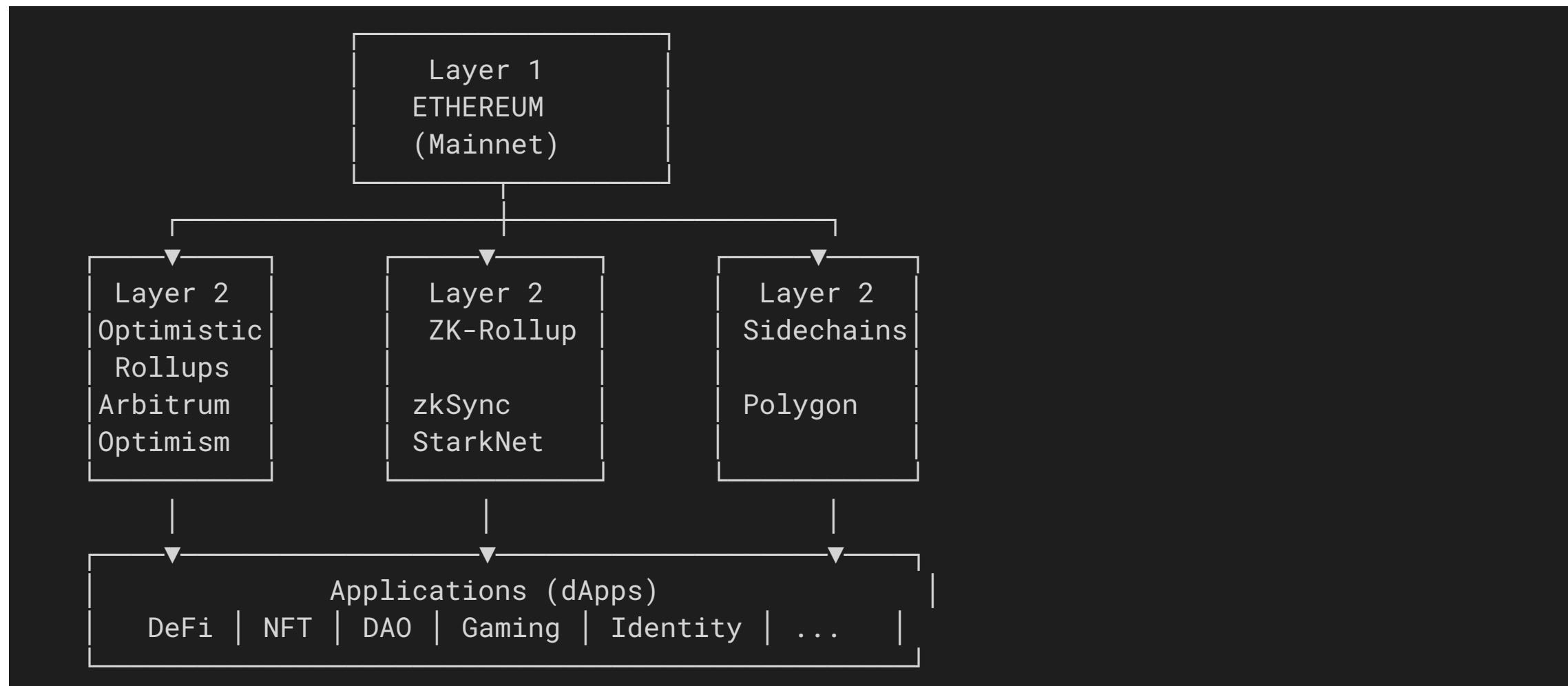
Différences clés avec Bitcoin :

Aspect	Bitcoin	Ethereum
Objectif	Monnaie numérique	Plateforme d'applications
Smart Contracts	Script limité	Solidity (Turing-complet)
Modèle	UTXO	Account-based
Temps de bloc	~10 min	~12 secondes
Consensus	PoW	PoS (depuis The Merge)
Gas	Frais fixes	Frais dynamiques (gas)
VM	-	EVM (Ethereum Virtual Machine)

“ Ethereum = "ordinateur mondial décentralisé" capable d'exécuter du code arbitraire.

”

L'écosystème Ethereum



Hyperledger Fabric – Blockchain d'entreprise

Caractéristiques :

- Blockchain **privée et permissionnée**
- Pas de cryptomonnaie native
- **Channels** : sous-réseaux privés entre participants choisis
- **Chaincode** : smart contracts en Go, Java ou JavaScript
- Architecture **modulaire** (consensus pluggable)

Hyperledger Fabric	
Orderer	Ordering Service (RAFT)
Peers	Endorsement + Commit
CA	Membership Service Provider
Channels	Isolation des données
Chaincode	Logique métier (Go/JS/Java)
Private Data	Collections confidentielles

Comparatif des plateformes

Critère	Bitcoin	Ethereum	Solana	Hyperledger
Type	Publique	Publique	Publique	Privée
Consensus	PoW	PoS	PoH + PoS	RAFT/PBFT
TPS	~7	~30	~65 000	~3 000+
Finalité	~60 min	~12 min	~400ms	~2s
Smart Contracts	Limité	Solidity	Rust	Go/JS/Java
Gas/Frais	~\$1-50	~\$0.5-50	~\$0.0002	Pas de frais
Cible	Monnaie	dApps	Haute perf.	Entreprise

Module 4

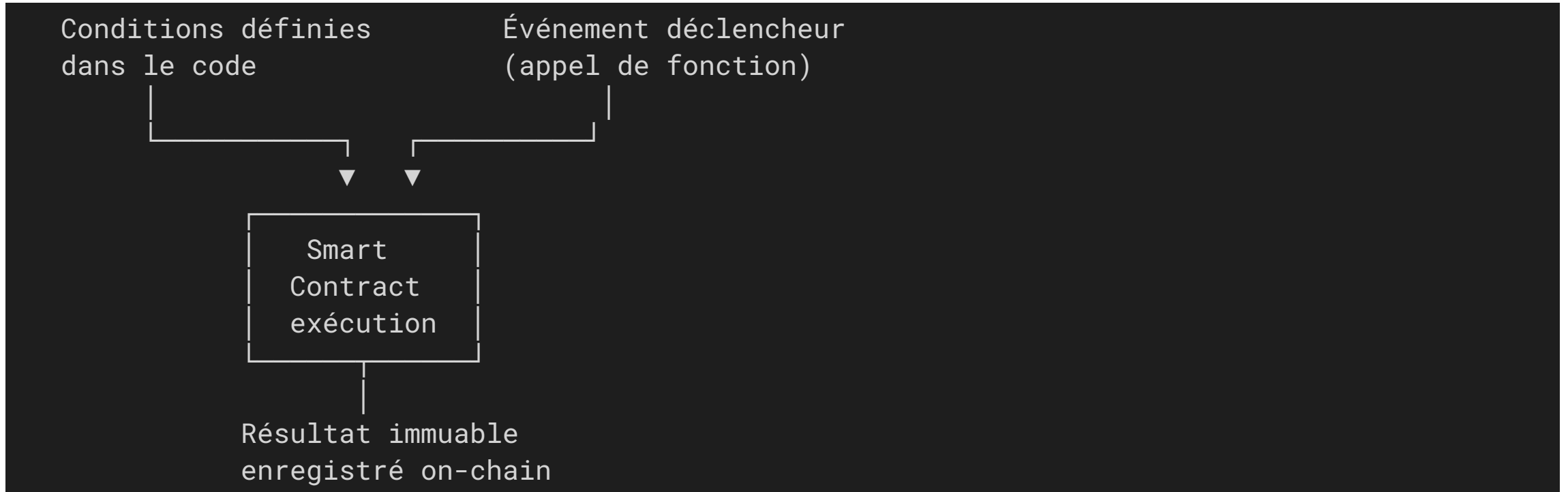
Smart Contracts

Qu'est-ce qu'un Smart Contract ?

Un **smart contract** est un programme stocké et exécuté sur la blockchain :

- **Auto-exécutable** : s'exécute automatiquement quand les conditions sont remplies
- **Immuable** : une fois déployé, le code ne peut plus être modifié
- **Déterministe** : même entrée = même résultat sur tous les nœuds
- **Transparent** : le code est visible et vérifiable par tous

Qu'est-ce qu'un Smart Contract ?



Solidity — Le langage d'Ethereum (1/2)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract SimpleStorage {
    // Variable d'état (stockée sur la blockchain)
    uint256 private storedValue;

    // Mapping : équivalent d'une HashMap
    mapping(address => uint256) public balances;

    // Événement (log sur la blockchain)
    event ValueChanged(address indexed author, uint256 newValue);

    // Modifieur : contrôle d'accès réutilisable
    modifier onlyPositive(uint256 value) {
        require(value > 0, "La valeur doit etre positive");
        _;
    }
}
```

Solidity – Le langage d'Ethereum (2/2)

```
// Fonction d'écriture (coûte du gas)
function set(uint256 newValue) public onlyPositive(newValue) {
    storedValue = newValue;
    balances[msg.sender] += 1;
    emit ValueChanged(msg.sender, newValue);
}

// Fonction de lecture (gratuite)
function get() public view returns (uint256) {
    return storedValue;
}
}
```

Smart Contract – Token ERC-20 (1/2)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract FormationToken is ERC20, Ownable {

    uint256 public constant MAX_SUPPLY = 1_000_000 * 10**18;

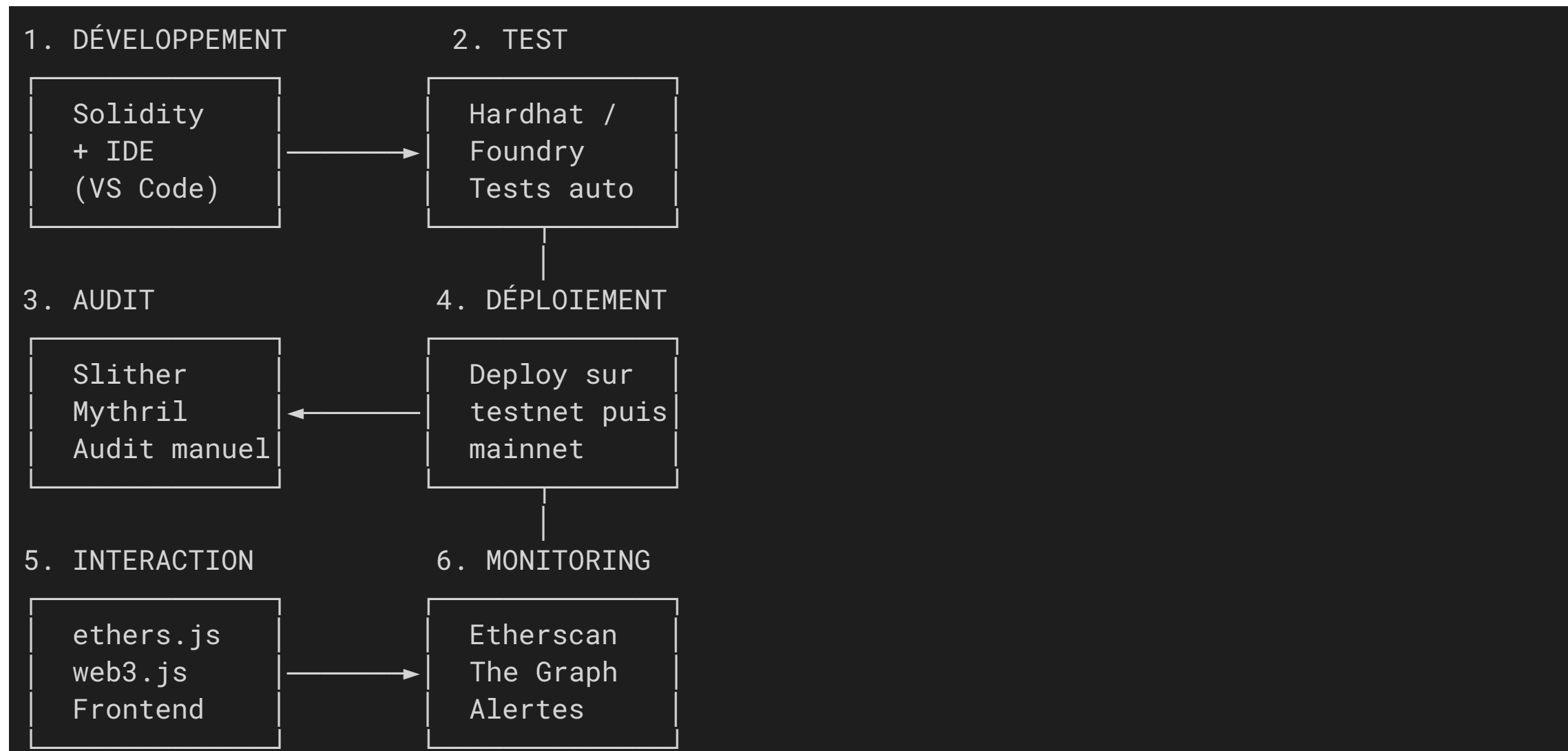
    constructor() ERC20("Formation Token", "FTK") Ownable(msg.sender) {
        // Mint initial supply au déployeur
        _mint(msg.sender, 100_000 * 10**18);
    }
}
```

Smart Contract – Token ERC-20 (2/2)

```
function mint(address to, uint256 amount) public onlyOwner {  
    require(totalSupply() + amount <= MAX_SUPPLY, "Max supply exceeded");  
    _mint(to, amount);  
}  
  
function burn(uint256 amount) public {  
    _burn(msg.sender, amount);  
}  
}
```

“ **ERC-20** : standard pour les tokens fongibles sur Ethereum (USDT, LINK, UNI...). ”

Cycle de vie d'un Smart Contract



Vulnérabilités des Smart Contracts

Vulnérabilité	Description	Exemple célèbre
Reentrancy	Rappel récursif avant mise à jour d'état	The DAO Hack (2016) – 60M\$
Integer Overflow	Dépassement de capacité numérique	BeautyChain (2018)
Access Control	Fonctions critiques non protégées	Parity Wallet (2017) – 150M\$
Front-running	Transaction visible en mempool	Attaques sandwich sur DEX
Oracle Manipulation	Données externes falsifiées	Mango Markets (2022) – 114M\$
Flash Loan Attack	Emprunt massif instantané	bZx, Cream Finance

Reentrancy Attack – Contrat vulnérable

```
// CONTRAT VULNÉRABLE
contract VulnerableBank {
    mapping(address => uint256) public balances;

    function withdraw() public {
        uint256 amount = balances[msg.sender];
        // ✗ Envoi AVANT la mise à jour du solde
        (bool success, ) = msg.sender.call{value: amount}("");
        require(success);
        balances[msg.sender] = 0; // Trop tard !
    }
}
```

Reentrancy Attack – Contrat corrigé

```
// CONTRAT CORRIGÉ (pattern Checks-Effects-Interactions)
contract SecureBank {
    mapping(address => uint256) public balances;

    function withdraw() public {
        uint256 amount = balances[msg.sender];
        require(amount > 0, "Solde insuffisant");
        balances[msg.sender] = 0; // ✅ Mise à jour AVANT l'envoi
        (bool success, ) = msg.sender.call{value: amount}("");
        require(success, "Transfert echoue");
    }
}
```

Bonnes pratiques de sécurité Smart Contracts

Pattern Checks-Effects-Interactions :

1. CHECKS : vérifier les conditions (require, assert)
2. EFFECTS : modifier l'état du contrat
3. INTERACTIONS : appeler des contrats externes

Bonnes pratiques de sécurité Smart Contracts

Règles essentielles :

- Utiliser OpenZeppelin pour les standards (ERC-20, ERC-721, Access Control)
- Appliquer le principe du moindre privilège
- Utiliser
`ReentrancyGuard`
pour les fonctions sensibles
- Préférer
`pull`
over
`push`
pour les paiements
- Limiter la complexité (gas limit)
- Auditer avec des outils automatisés (Slither, Mythril) ET manuellement
- Déployer sur testnet avant mainnet
- Implémenter un pattern de pause d'urgence (
`Pausable`
)

Module 5

Applications Sectorielles

Vue d'ensemble des applications

APPLICATIONS BLOCKCHAIN			
Finance (DeFi)	Supply Chain	Identité (SSI)	Gouvernance (DAO)
Prêts Échanges Assurance Stables Paielements	Traçabi- lité Origine Anti- contrefaçon	Diplômes Passeport KYC Dossier médical	Vote décentralisé Trésorerie collective Propositions
NFT & Tokenisa- tion	Énergie P2P trading	Immobilier Fractional ownership	Secteur public Registres fonciers

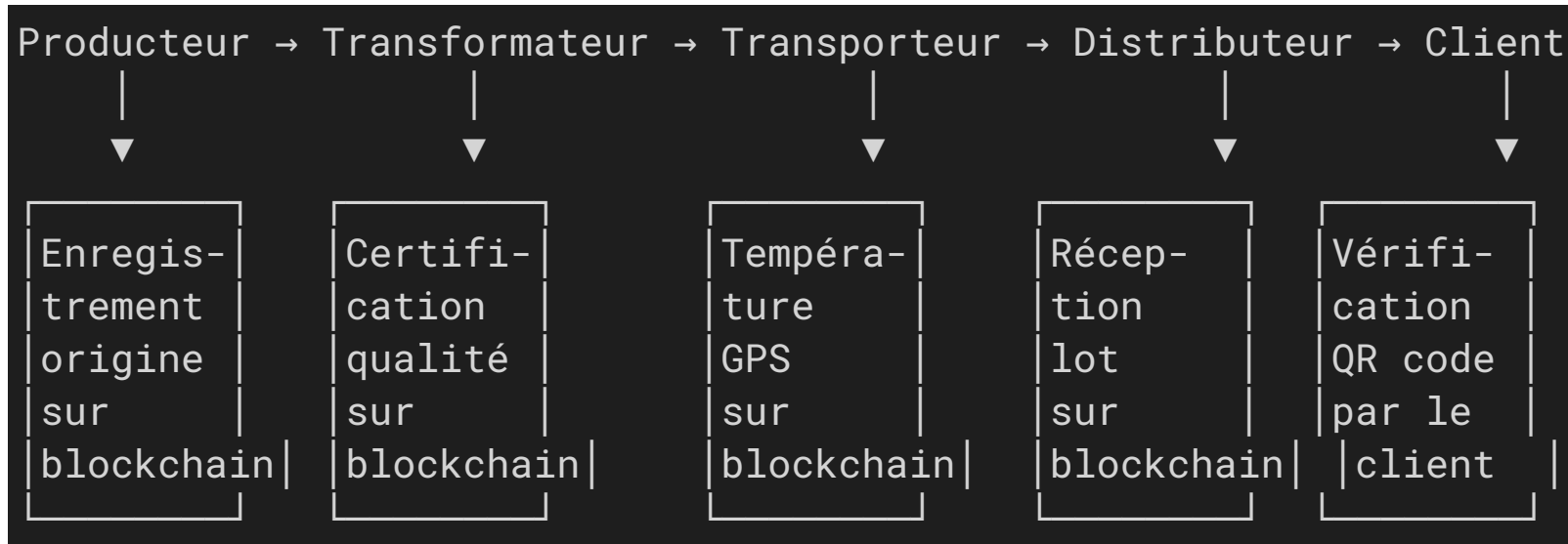
DeFi — Finance Décentralisée

DeFi reproduit les services financiers traditionnels sans intermédiaire :

Service traditionnel	Équivalent DeFi	Protocole exemple
Banque (dépôts)	Lending/Borrowing	Aave, Compound
Bourse (échange)	DEX (échange décentralisé)	Uniswap, Curve
Produits dérivés	Derivatives	dYdX, GMX
Assurance	Insurance pools	Nexus Mutual
Paielements	Stablecoins	USDC, DAI
Gestion d'actifs	Yield aggregators	Yearn Finance

“ **TVL** (Total Value Locked) dans la DeFi : ~80 milliards \$ (2024). ”

Supply Chain – Traçabilité

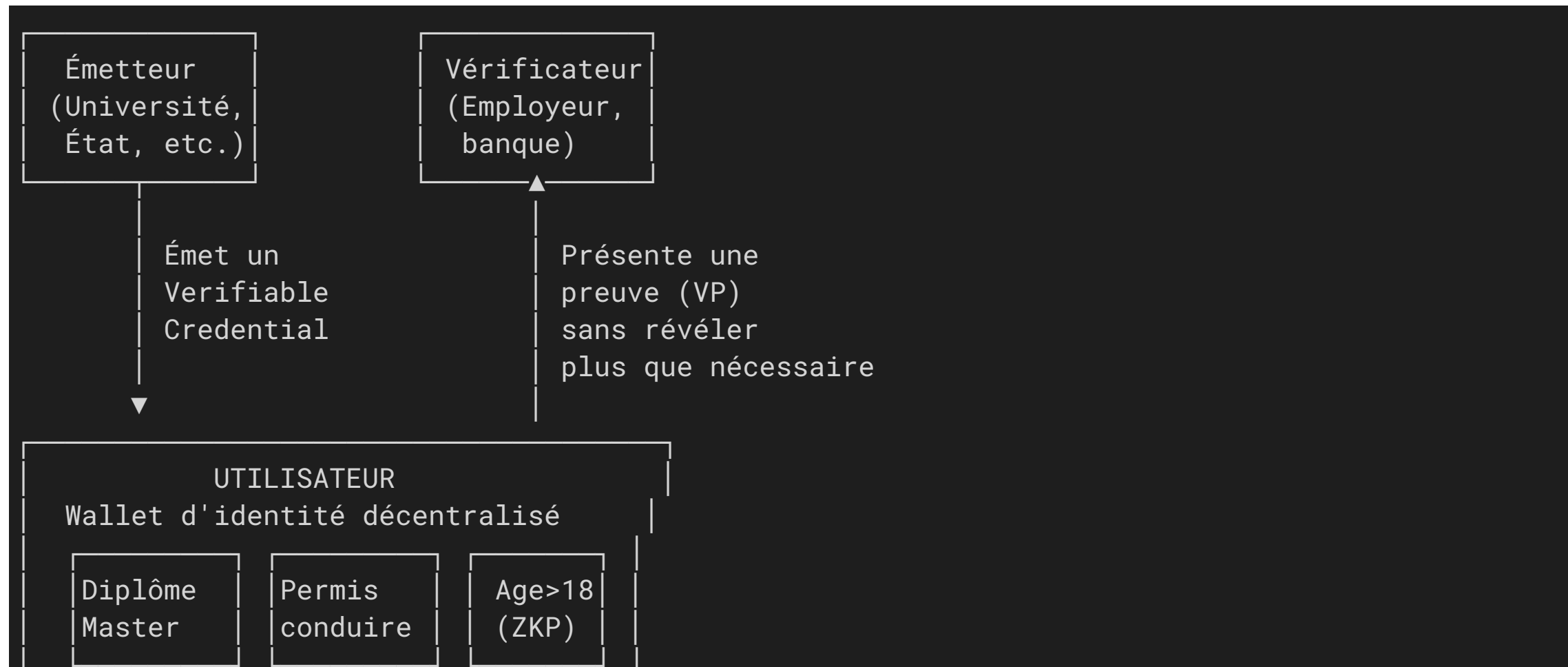


Exemples réels :

- **Walmart** : traçabilité alimentaire (IBM Food Trust)
- **De Beers** : traçabilité des diamants (Tracr)
- **Maersk** : commerce international (TradeLens)
- **LVMH** : authentification luxe (Aura)

Identité Décentralisée (SSI)

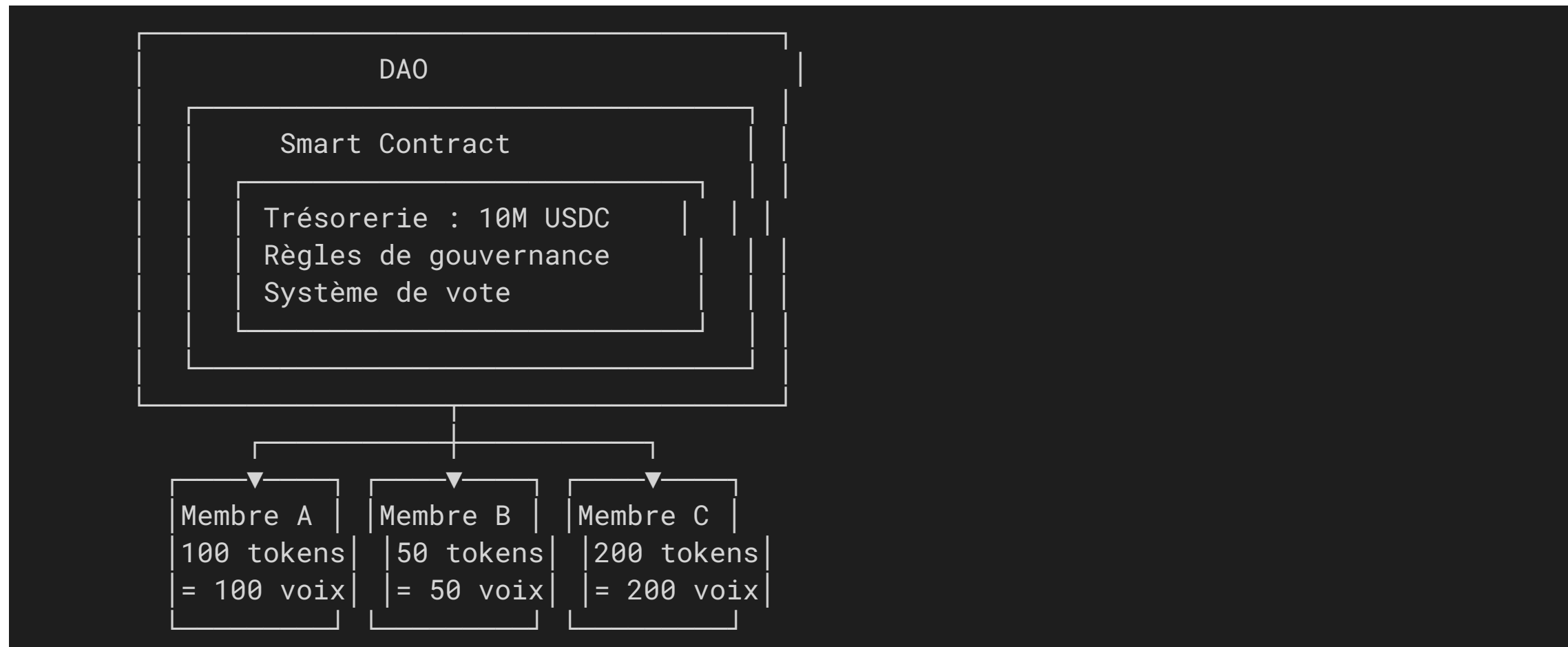
Self-Sovereign Identity : l'utilisateur contrôle son identité



Identité Décentralisée (SSI)

“ **Zero-Knowledge Proof** : prouver que j'ai plus de 18 ans sans révéler ma date de naissance. ”

DAO – Organisation Autonome Décentralisée



Exemples : MakerDAO (stablecoin DAI), Uniswap (DEX), ENS (noms de domaine), Gitcoin (financement open source).

Tokenisation des actifs réels (RWA)

Real World Assets tokenisés sur la blockchain :

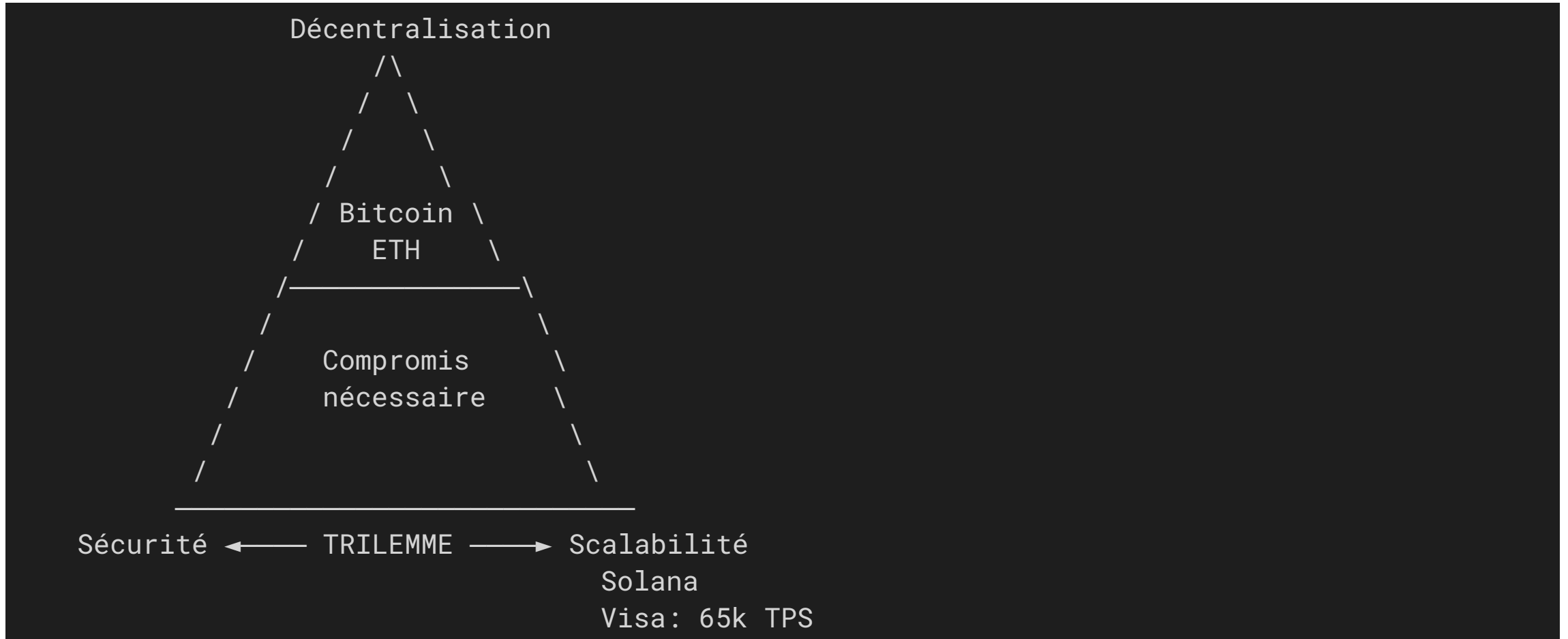
Actif	Avantage de la tokenisation
Immobilier	Propriété fractionnée, liquidité accrue
Art	Accès démocratisé, provenance vérifiable
Obligations	Règlement instantané, transparence
Matières premières	Traçabilité, échanges 24/7
Actions	Settlement T+0 au lieu de T+2
Crédits carbone	Transparence, anti-double-comptage

“ **Projection** : le marché des RWA tokenisés pourrait atteindre 16 000 milliards \$ d'ici 2030 (Boston Consulting Group). ”

Module 6

Défis et Limitations

Le trilemme de la Blockchain



“ Toute blockchain doit faire des compromis entre ces trois propriétés. Aucune ne maximise les trois simultanément.

Scalabilité – Solutions

Approche	Description	Exemples
Layer 1	Améliorer la blockchain de base	Sharding (Ethereum), bigger blocks
Layer 2 Rollups	Exécuter off-chain, prouver on-chain	Arbitrum, Optimism, zkSync
Sidechains	Chaînes parallèles liées	Polygon PoS
State Channels	Transactions off-chain bilatérales	Lightning Network (Bitcoin)
Validium	Données off-chain, preuves on-chain	StarkEx

Rollups expliqués :

- **Optimistic** : suppose que les transactions sont valides, période de contestation (7 jours)
- **ZK (Zero-Knowledge)** : prouve mathématiquement la validité, finalité rapide

Consommation énergétique

Réseau	Consensus	Consommation annuelle
Bitcoin	PoW	~150 TWh (comparable à la Pologne)
Ethereum (avant Merge)	PoW	~78 TWh
Ethereum (après Merge)	PoS	~0.01 TWh (-99.95%)
Solana	PoH + PoS	~0.005 TWh
Visa	Centralisé	~0.2 TWh

“ Le passage d'Ethereum au Proof of Stake a réduit sa consommation de 99.95%.

”

Nuances :

- Le minage Bitcoin utilise ~59% d'énergie renouvelable (Cambridge, 2023)
- Le PoS élimine le problème mais introduit des risques de centralisation des validateurs

Enjeux réglementaires

Région	Approche	Réglementation clé
UE	Encadrement structuré	MiCA (Markets in Crypto-Assets) 2024
USA	Fragmentée (SEC vs CFTC)	Débat security vs commodity
Chine	Interdiction crypto, pro-CBDC	Yuan numérique (e-CNY)
Suisse	Crypto-friendly	"Crypto Valley" Zug
Singapour	Régulation progressive	MAS framework

Défis RGPD vs Blockchain :

- **Droit à l'oubli** vs **immuabilité** de la blockchain
- **Responsable de traitement** difficile à identifier sur réseau décentralisé
- Solutions : stockage off-chain + hash on-chain, Zero-Knowledge Proofs

Limites techniques actuelles

Stockage :

- Blockchain Bitcoin : ~500 Go et croissant
- Blockchain Ethereum : ~1 To (full node), ~13 To (archive)
- Coût de stockage on-chain : ~\$20k par Mo sur Ethereum

Interopérabilité :

- Les blockchains sont des silos isolés
- Solutions : bridges cross-chain (mais vulnérables – \$2B piratés en 2022)
- Standards émergents : IBC (Cosmos), XCM (Polkadot)

Vie privée :

- Blockchain publique = pseudonyme, pas anonyme
- Analyse on-chain peut désanonymiser (Chainalysis, Elliptic)
- Solutions : Zcash (zk-SNARKs), Monero (Ring Signatures), Tornado Cash

L'avenir de la Blockchain

Tendances 2024-2030 :

- **Account Abstraction** (ERC-4337) : UX simplifiée, plus besoin de gérer des clés privées manuellement
- **Zero-Knowledge Proofs** : scalabilité ET confidentialité
- **CBDC** (Central Bank Digital Currency) : monnaies numériques de banque centrale
- **RWA Tokenisation** : actions, obligations, immobilier sur la blockchain
- **DePIN** (Decentralized Physical Infrastructure) : Filecoin, Helium, Render
- **AI + Blockchain** : agents autonomes avec portefeuilles, données d'entraînement vérifiables
- **Interopérabilité native** : communication transparente entre chaînes
- **Adoption institutionnelle** : BlackRock, JPMorgan, Visa intègrent la blockchain